Modern Cryptography and AI/ML Security

LECTURE 32

Lattice-based Cryptography

IISER THIRUVANANTHAPURAM

Course ID: DSC 4110/6111 **Instructor:** Dr. Laltu Sardar **Date:** November 2, 2026

1 Lattice-based Cryptography

Cryptography allows two or more people who are geographically far apart to communicate secretly. However, any communication medium can be vulnerable to attacks, and therefore, we need cryptographic techniques to protect the confidentiality, integrity, and authenticity of information.

In a Public Key Infrastructure (PKI), symmetric encryption algorithms, such as stream ciphers and block ciphers, can be used to protect the actual communication. Hash functions and digital signatures can provide data integrity, authenticity, and public-key verification. However, symmetric encryption requires both communicating parties to share a common secret key securely. Establishing this shared secret key is a major challenge, and this is where Public Key Encryption (PKE) becomes important.

Unfortunately, most efficient classical public-key encryption schemes, such as RSA and Elliptic Curve Cryptography (ECC), rely on the hardness of integer factorization and the Discrete Logarithm Problem (DLP). These problems can be solved efficiently using sufficiently powerful quantum computers, making such cryptographic schemes unsuitable for the quantum era. Therefore, we need alternative cryptographic techniques that can resist quantum attacks. Code-based cryptography is one such approach that we have studied earlier. Another promising approach is Lattice-Based Cryptography, which provides efficient and quantum-resistant public-key encryption constructions. In this lecture note, we study the basic concepts of lattices and their applications in cryptography, keeping the mathematical and technical details as simple as possible.

1.1 Vector Space

A **vector space** is a collection of vectors in which vector addition and scalar multiplication are defined and satisfy certain mathematical properties. For example, $\mathbb{R}^2$, the set of all two-dimensional real vectors, forms a vector space. Vector spaces provide the mathematical foundation for understanding lattices.

Vector Space

Let $\mathbb{F}$ be a field and V be a non-empty set. Then V is a vector space over $\mathbb{F}$ if it supports vector addition and scalar multiplication satisfying the following properties for all $u, v, w \in V$ and $a, b \in \mathbb{F}$:

1. $u + v = v + u$ (Commutativity)
2. $(u + v) + w = u + (v + w)$ (Associativity)
3. There exists $0 \in V$ such that $v + 0 = v$ (Additive identity)
4. For every $v \in V$, there exists $-v \in V$ such that $v + (-v) = 0$ (Additive inverse)
5. $a(u + v) = au + av$ (Distributivity)
6. $(a + b)v = av + bv$ (Distributivity)
7. $a(bv) = (ab)v$ (Scalar associativity)
8. $1v = v$ (Scalar identity)

1.2 Linear Combination and Linear Span

Let V be a vector space over a field $\mathbb{F}$, and let $B = \{v_1, v_2, \ldots, v_m\}$ be a collection of vectors in V. A **linear combination** of these vectors is an expression of the form

$$v = a_1 v_1 + a_2 v_2 + \cdots + a_m v_m,$$

where $a_1, a_2, \ldots, a_m \in \mathbb{F}$.

The set of all possible linear combinations of the vectors in B is called their **linear span**.

Linear Span

The linear span of a set of vectors $B = \{v_1, v_2, \ldots, v_m\}$ is defined as

$$\operatorname{span}_{\mathbb{F}}(B) = \left\{ \sum_{i=1}^{m} a_i v_i \;\middle|\; a_i \in \mathbb{F} \right\}.$$

For example, consider two vectors $v_1 = (1, 0)$ and $v_2 = (0, 1)$ in $\mathbb{R}^2$. Their linear span is

$$\operatorname{span}_{\mathbb{R}}\{v_1, v_2\} = \mathbb{R}^2.$$

This is because every vector $(x, y) \in \mathbb{R}^2$ can be expressed as

$$(x, y) = x(1, 0) + y(0, 1).$$

In the context of lattices, we will use a similar idea, but restrict the coefficients to integers rather than arbitrary real numbers. This leads to the definition of a lattice.

1.3 Lattice Definition

Lattice

Let $\boldsymbol{b}_1, \ldots, \boldsymbol{b}_n \in \mathbb{R}^m$ be linearly independent vectors, where $n \leq m$. The lattice generated by these vectors is defined as

$$\boxed{\mathcal{L}(B) = \left\{ \sum_{i=1}^{n} z_i \boldsymbol{b}_i : z_i \in \mathbb{Z} \right\},}$$

where $B = [\boldsymbol{b}_1\ \boldsymbol{b}_2\ \cdots\ \boldsymbol{b}_n] \in \mathbb{R}^{m \times n}$ is called the basis matrix. Equivalently,

$$\mathcal{L}(B) = \{B\boldsymbol{z} : \boldsymbol{z} \in \mathbb{Z}^n\}.$$

The coefficients must be integers. Thus, a lattice consists of discrete points rather than all points in the real span of its basis vectors.

A vector space over $\mathbb{R}$ allows arbitrary real-valued linear combinations of its basis vectors. A lattice, in contrast, is a discrete set of points obtained using integer-valued combinations of linearly independent vectors. This distinction is fundamental in lattice-based cryptography.

Remark

The rank of a lattice is n, the number of linearly independent basis vectors. Its ambient dimension is m. A full-rank lattice in $\mathbb{R}^m$ has rank m.

1.4 A two-dimensional example

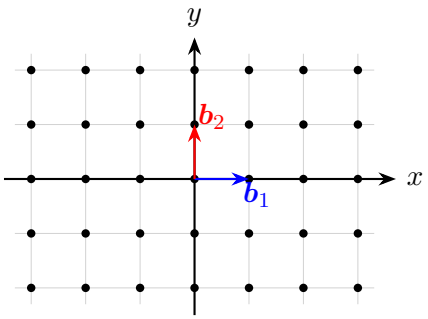
Consider

$$\boldsymbol{b}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad \boldsymbol{b}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Then

$$\mathcal{L}(B) = \left\{ \begin{pmatrix} z_1 \\ z_2 \end{pmatrix} : z_1, z_2 \in \mathbb{Z} \right\} = \mathbb{Z}^2.$$

This is the familiar square grid of integer-coordinate points.



Every lattice point has the form

$$\boldsymbol{x} = z_1\boldsymbol{b}_1 + z_2\boldsymbol{b}_2, \qquad z_1, z_2 \in \mathbb{Z}.$$

For example, $(2, -1)^T = 2\boldsymbol{b}_1 - \boldsymbol{b}_2$ is a lattice point, whereas $(1/2, 1/3)^T$ is not.

1.5 A Lattice is Infinite but Discrete

A lattice contains infinitely many points. For example, for any nonzero basis vector $\boldsymbol{b}_1$, all integer multiples

$$\ldots, -2\boldsymbol{b}_1, -\boldsymbol{b}_1, \boldsymbol{0}, \boldsymbol{b}_1, 2\boldsymbol{b}_1, \ldots$$

belong to the lattice.

However, a lattice does not fill its real span continuously. There is a positive minimum distance between any two distinct lattice points. This distance is called the minimum distance of the lattice and is defined as

$$\lambda_1(\mathcal{L}) = \min_{\substack{\boldsymbol{v}\in\mathcal{L} \\ \boldsymbol{v}\neq\boldsymbol{0}}} \|\boldsymbol{v}\|_2.$$

For every lattice, $\lambda_1(\mathcal{L}) > 0$. Thus, a lattice is an infinite but discrete set of points.

2 Bases of a Lattice

A lattice can have infinitely many different bases, even though the lattice itself remains unchanged. Let us consider an example.

Example

Consider the following two matrices:

$$B = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad B' = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}.$$

The columns of B' are

$$\boldsymbol{b}'_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad \boldsymbol{b}'_2 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}.$$

For any $(x, y)^T \in \mathbb{Z}^2$, we have

$$\begin{pmatrix} x \\ y \end{pmatrix} = (x - 2y)\boldsymbol{b}'_1 + y\boldsymbol{b}'_2,$$

where $x - 2y, y \in \mathbb{Z}$. Thus, every point in $\mathbb{Z}^2$ can be represented as an integer combination of the columns of B'. Consequently,

$$\mathcal{L}(B) = \mathcal{L}(B') = \mathbb{Z}^2.$$

The above example demonstrates that different basis matrices can generate the same lattice. This raises an important question:

What is the mathematical relationship between two different basis matrices that generate the same lattice? Given one basis, can we systematically generate another basis for the same lattice?

To answer this question, we first introduce the concept of **unimodular matrices**.

2.1 Unimodular Matrices

Unimodular Matrix

A square integer matrix $U \in \mathbb{Z}^{n\times n}$ is called unimodular if

$$\det(U) = +1 \quad \text{or} \quad \det(U) = -1.$$

Equivalently, U^{-1} is also an integer matrix.

Let us now see how unimodular matrices relate different bases of the same lattice.

Theorem 2.1 (Change of Basis). *Let $B \in \mathbb{R}^{m\times n}$ have linearly independent columns and let $U \in \mathbb{Z}^{n\times n}$ be unimodular. Then*

$$\boxed{\mathcal{L}(BU) = \mathcal{L}(B).}$$

Conversely, any two basis matrices B and B' of the same rank-n lattice are related by a unimodular integer matrix, i.e.,

$$B' = BU$$

for some unimodular matrix $U \in \mathbb{Z}^{n\times n}$.

Proof. First, since U is an integer matrix, $U\boldsymbol{z} \in \mathbb{Z}^n$ for every $\boldsymbol{z} \in \mathbb{Z}^n$. Therefore,

$$\begin{aligned}\mathcal{L}(BU) &= \{BU\boldsymbol{z} : \boldsymbol{z} \in \mathbb{Z}^n\}\\ &\subseteq \{B\boldsymbol{y} : \boldsymbol{y} \in \mathbb{Z}^n\}\\ &= \mathcal{L}(B).\end{aligned}$$

Since U is unimodular, U^{-1} is also an integer matrix. Applying the same argument to U^{-1} gives

$$\mathcal{L}(B) \subseteq \mathcal{L}(BU).$$

Hence, $\mathcal{L}(BU) = \mathcal{L}(B)$.

For the converse, suppose B and B' generate the same lattice. Since every column of B' belongs to $\mathcal{L}(B)$, there exists an integer matrix $U \in \mathbb{Z}^{n\times n}$ such that $B' = BU$.

Similarly, since every column of B belongs to $\mathcal{L}(B')$, there exists an integer matrix $V \in \mathbb{Z}^{n\times n}$ such that $B = B'V$.

Combining these equations gives

$$B = BUV.$$

Since the columns of B are linearly independent, $UV = I_n$. Thus, $U^{-1} = V \in \mathbb{Z}^{n\times n}$. Consequently, $\det(U) = \pm 1$, and hence U is unimodular. □

Thus, given any basis of a lattice, we can generate other bases by multiplying it with unimodular matrices. A change of basis changes the coordinate description and the geometry of the displayed basis vectors, but it does not change the lattice points.

Use of different bases in PKE Although unimodular transformations preserve the lattice, they do not necessarily preserve the lengths or angles of the basis vectors. Consequently, two bases of the same lattice may have significantly different geometric properties.

Example

Consider the two basis matrices:

$$B = \begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix}, \qquad B' = \begin{pmatrix}1 & 2\\ 0 & 1\end{pmatrix}.$$

Both generate the same lattice $\mathbb{Z}^2$. However, their basis vectors have different lengths:

$$\begin{aligned}\|\boldsymbol{b}_1\|_2 &= 1, \qquad \|\boldsymbol{b}_2\|_2 = 1,\\ \|\boldsymbol{b}'_1\|_2 &= 1, \qquad \|\boldsymbol{b}'_2\|_2 = \sqrt{5}.\end{aligned}$$

Thus, although both bases generate exactly the same lattice, their geometric properties are different.

3 Computational Problems in Lattices

In lattice-based cryptography, the choice of basis can significantly affect the efficiency of lattice algorithms. A basis consisting of relatively short and nearly orthogonal vectors is generally more suitable for computation than a basis containing long and highly correlated vectors. Finding a good basis from a given basis is therefore an important computational problem.

Lattices have several important computational problems that are believed to be difficult to solve, particularly in high dimensions. These problems involve finding specific lattice vectors satisfying certain geometric or algebraic conditions. Although a lattice can be efficiently represented using a basis matrix, finding particular vectors within the lattice may require significant computational effort.

The difficulty of these problems forms the foundation of lattice-based cryptography. In particular, several post-quantum cryptographic constructions rely on the computational hardness of lattice problems, which are believed to remain difficult even for quantum computers. Depending on the objective, these problems can be classified into different variants, such as finding the shortest nonzero lattice vector, finding a lattice vector closest to a given target, or finding a short integer vector satisfying a system of modular linear equations.

In this section, we study some fundamental computational problems in lattices, their different variants, and their relationships. We begin with the Shortest Vector Problem (SVP).

3.1 Shortest Vector Problem (SVP)

One of the fundamental computational problems in lattice theory is to find the shortest nonzero vector in a given lattice. Although a lattice contains infinitely many vectors, there exists a nonzero vector having the minimum Euclidean length. Finding such a vector is known as the Shortest Vector Problem.

First Successive Minimum

For a lattice $\mathcal{L} \subset \mathbb{R}^m$, its first successive minimum is defined as

$$\lambda_1(\mathcal{L}) = \min_{\boldsymbol{v} \in \mathcal{L} \setminus \{\mathbf{0}\}} \|\boldsymbol{v}\|_2 .$$

It represents the Euclidean length of a shortest nonzero vector in the lattice.

Shortest Vector Problem (SVP)

Given a basis B of a lattice $\mathcal{L}(B)$, find a nonzero vector $\boldsymbol{v} \in \mathcal{L}(B)$ such that

$$\boxed{\|\boldsymbol{v}\|_2 = \lambda_1(\mathcal{L}(B)).}$$

This is called the exact Shortest Vector Problem (SVP).

Example Consider the standard lattice $\mathbb{Z}^2$, generated by

$$B = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Some of its lattice points are

$$\mathcal{L}(B) = \{(x, y)^T : x, y \in \mathbb{Z}\}.$$

The shortest nonzero lattice vectors are

$$(1, 0)^T, \quad (-1, 0)^T, \quad (0, 1)^T, \quad (0, -1)^T.$$

Each has Euclidean length 1. Therefore,

$$\boxed{\lambda_1(\mathbb{Z}^2) = 1.}$$

An important observation The shortest nonzero lattice vector is not necessarily one of the basis vectors. Consider the basis

$$B = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}.$$

Its basis vectors are

$$\boldsymbol{b}_1 = (2,0)^T, \qquad \boldsymbol{b}_2 = (1,1)^T.$$

However, their difference gives

$$\boldsymbol{b}_1 - 2\boldsymbol{b}_2 = (0,-2)^T,$$

which is not shorter than $\boldsymbol{b}_2$. More generally, the shortest lattice vector must be found among all nonzero integer combinations of the basis vectors, not merely among the basis vectors themselves.

3.2 Approximate SVP

Finding the exact shortest nonzero vector in a lattice is computationally difficult, particularly in high dimensions. Currently, no polynomial-time algorithm is known for solving exact SVP for arbitrary lattices. In fact, exact SVP is known to be NP-hard under randomized reductions. Therefore, directly using exact SVP as a computational problem in practical cryptographic constructions is not always convenient.

Instead of finding the absolute shortest vector, we can relax the requirement and search for a sufficiently short vector whose length is bounded by a factor of the shortest possible length. This leads to the approximate version of SVP.

γ- SVP

Let $\gamma \geq 1$. Given a basis B of a lattice $\mathcal{L}(B)$, find a nonzero vector $\boldsymbol{v} \in \mathcal{L}(B)$ satisfying

$$\boxed{\|\boldsymbol{v}\|_2 \leq \gamma \lambda_1(\mathcal{L}(B)).}$$

The problem is called γ-Approximate SVP.

Here, γ is called the approximation factor. When $\gamma = 1$, the problem is exactly SVP. For $\gamma > 1$, the solution need not be the shortest vector, but its length must not exceed γ times the length of the shortest nonzero lattice vector.

For example, if $\lambda_1(\mathcal{L}) = 5$ and $\gamma = 2$, any nonzero lattice vector of length at most 10 is an acceptable solution to 2-Approximate SVP.

The LLL algorithm, introduced earlier, provides a polynomial-time algorithm for finding an approximate solution to SVP. Its approximation factor is exponential in the lattice dimension. Although it does not solve exact SVP in polynomial time, it provides a useful approximation.

4 The Dual Lattice

The dual lattice is an important mathematical concept that provides a complementary representation of a given lattice. While the original lattice consists of integer combinations of its basis vectors, the dual lattice consists of vectors whose inner products with every vector of the original lattice are integers.

Dual Lattice

Let $\mathcal{L} \subset \mathbb{R}^m$ be a full-rank lattice. Its dual lattice is defined as

$$\boxed{\mathcal{L}^* = \{\boldsymbol{y} \in \mathbb{R}^m : \langle \boldsymbol{y}, \boldsymbol{x} \rangle \in \mathbb{Z} \text{ for every } \boldsymbol{x} \in \mathcal{L}\}.}$$

4.1 Basis of the Dual Lattice

If $B \in \mathbb{R}^{m \times m}$ is a basis matrix of a full-rank lattice, then the basis matrix of its dual lattice is given by

$$\boxed{B^* = (B^{-1})^T = B^{-T}.}$$

Consequently,

$$\boxed{\mathcal{L}(B)^* = B^{-T}\mathbb{Z}^m.}$$

Proof. A vector $\boldsymbol{y}$ belongs to $\mathcal{L}(B)^*$ exactly when

$$\langle \boldsymbol{y}, B\boldsymbol{z} \rangle \in \mathbb{Z} \quad \text{for every } \boldsymbol{z} \in \mathbb{Z}^m.$$

Since

$$\langle \boldsymbol{y}, B\boldsymbol{z} \rangle = \boldsymbol{y}^T B\boldsymbol{z} = (B^T\boldsymbol{y})^T\boldsymbol{z},$$

the above condition is equivalent to

$$B^T\boldsymbol{y} \in \mathbb{Z}^m.$$

Therefore, there exists $\boldsymbol{z} \in \mathbb{Z}^m$ such that

$$\boldsymbol{y} = B^{-T}\boldsymbol{z}.$$

Hence,

$$\mathcal{L}(B)^* = B^{-T}\mathbb{Z}^m.$$

□

Example

Consider the lattice $\mathcal{L} = \mathbb{Z}^2$ with basis

$$B = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Its dual basis is

$$B^* = (B^{-1})^T = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Thus,

$$\boxed{\mathcal{L}^* = \mathbb{Z}^2 = \mathcal{L}.}$$

The standard integer lattice is therefore self-dual.

Example

Consider a lattice generated by

$$B = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}.$$

Its dual basis matrix is

$$B^* = (B^{-1})^T = \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{3} \end{pmatrix}.$$

Therefore,

$$\mathcal{L}^* = \left\{ \begin{pmatrix} \frac{z_1}{2} \\ \frac{z_2}{3} \end{pmatrix} : z_1, z_2 \in \mathbb{Z} \right\}.$$

Notice that the dual lattice has points that are more closely spaced than those of the original lattice.

4.2 Properties of Dual Lattices

1. **Dual of the Dual:** For any full-rank lattice $\mathcal{L}$,

   $$\boxed{(\mathcal{L}^*)^* = \mathcal{L}.}$$

   For example, if $\mathcal{L} = 2\mathbb{Z}^2$, then $\mathcal{L}^* = \frac{1}{2}\mathbb{Z}^2$ and $(\mathcal{L}^*)^* = 2\mathbb{Z}^2 = \mathcal{L}$.

2. **Dual of a Scaled Lattice:** For any nonzero scalar $c \in \mathbb{R}$,

   $$\boxed{(c\mathcal{L})^* = \frac{1}{c}\mathcal{L}^*.}$$

For example, since $(\mathbb{Z}^2)^* = \mathbb{Z}^2$,

$$(3\mathbb{Z}^2)^* = \frac{1}{3}\mathbb{Z}^2.$$

3. **Determinant of the Dual:** For any full-rank lattice $\mathcal{L}$,

$$\boxed{\det(\mathcal{L}^*) = \frac{1}{\det(\mathcal{L})}.}$$

For example, if $B = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$, then $\det(\mathcal{L}) = 6$, whereas $B^* = \begin{pmatrix} 1/2 & 0 \\ 0 & 1/3 \end{pmatrix}$ gives $\det(\mathcal{L}^*) = 1/6$.

4.3 Why are Dual Lattices Important?

One important motivation for studying dual lattices is that they provide a complementary view of the geometric properties of the original lattice. In particular, the lengths of short vectors in a lattice are related to the lengths of vectors in its dual. Such relationships are captured by transference theorems, which connect the successive minima of a lattice and its dual. These relationships are useful in analysing lattice problems and their computational hardness.

5 q-ary Lattices

In practical cryptographic implementations, computations are often performed using modular arithmetic rather than unrestricted integer or real arithmetic. This is convenient because modular arithmetic operates over a finite set of residues, namely $\mathbb{Z}_q = \{0, 1, \ldots, q-1\}$, where $q \geq 2$. Thus, values remain within a bounded range, and arithmetic operations can be efficiently implemented using computers.

Lattice-based cryptography frequently uses this modular arithmetic to define structured lattices. Although the matrix operations are performed modulo q, the resulting lattices consist of integer vectors. Such lattices are called q-ary lattices and play an important role in constructions based on LWE and SIS.

Let $q \geq 2$ be an integer, usually chosen prime in basic presentations, and let

$$A \in \mathbb{Z}_q^{n \times m}.$$

Here, $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$ and all matrix-vector products involving A are computed modulo q.

The q-ary Kernel Lattice

The q-ary lattice associated with the modular matrix A is

$$\boxed{\Lambda_q^{\perp}(A) = \{\boldsymbol{x} \in \mathbb{Z}^m : A\boldsymbol{x} = \mathbf{0} \pmod q\}.}$$

It consists of all integer vectors mapped to zero by A modulo q.

The q-ary Image Lattice

The companion q-ary lattice is

$$\boxed{\Lambda_q(A) = \{\boldsymbol{A}^T\boldsymbol{s} + q\boldsymbol{z} : \boldsymbol{s} \in \mathbb{Z}^n,\ \boldsymbol{z} \in \mathbb{Z}^m\}.}$$

Equivalently, $\boldsymbol{s}$ may be taken modulo q, since changing $\boldsymbol{s}$ by a multiple of q changes $\boldsymbol{A}^T\boldsymbol{s}$ by a vector in $q\mathbb{Z}^m$.

Theorem 5.1 (Duality of q-ary Lattices)**.** *With the definitions above,*

$$\boxed{\Lambda_q(A)^* = \frac{1}{q}\Lambda_q^{\perp}(A).}$$

Equivalently,

$$\boxed{\Lambda_q^{\perp}(A)^* = \frac{1}{q}\Lambda_q(A).}$$

Remark

The transpose in $A^T \boldsymbol{s}$ is necessary for the dimensions to match: A is an $n \times m$ matrix, so $A^T \boldsymbol{s}$ is an m-dimensional vector.

5.1 A Small Modular Example

Let $q = 5$ and consider

$$A = (1 \quad 2) \in \mathbb{Z}_5^{1\times 2}.$$

The corresponding kernel lattice is

$$\Lambda_5^{\perp}(A) = \left\{ \binom{x_1}{x_2} \in \mathbb{Z}^2 : x_1 + 2x_2 = 0 \pmod 5 \right\}.$$

For example, $(1, 2)^T$ belongs to the kernel lattice because

$$1 + 2(2) = 5 = 0 \pmod 5.$$

More generally, the modular equation implies

$$x_1 + 2x_2 = 5k, \qquad k \in \mathbb{Z}.$$

Therefore,

$$x_1 = 5k - 2x_2.$$

Writing $k = z_1$ and $x_2 = z_2$, we obtain

$$\binom{x_1}{x_2} = z_1 \binom{5}{0} + z_2 \binom{-2}{1}.$$

Hence, a basis matrix is

$$\boxed{B_{\perp} = \begin{pmatrix} 5 & -2 \\ 0 & 1 \end{pmatrix}.}$$

Similarly, the corresponding image lattice is

$$\Lambda_5(A) = \left\{ \begin{pmatrix} s + 5z_1 \\ 2s + 5z_2 \end{pmatrix} : s, z_1, z_2 \in \mathbb{Z} \right\}.$$

A basis matrix for this lattice is

$$\boxed{B = \begin{pmatrix} 1 & 0 \\ 2 & 5 \end{pmatrix}.}$$

Notice that

$$B^{-T} = \begin{pmatrix} 5 & -2 \\ 0 & 1 \end{pmatrix} = B_{\perp}.$$

Therefore,

$$\boxed{\Lambda_5(A)^* = \frac{1}{5}\Lambda_5^{\perp}(A).}$$

This illustrates the duality relationship between the two q-ary lattices.

5.2 Computing Bases of q-ary Lattices.

Unlike the general lattice representation, where a lattice is defined using a basis matrix B, a q-ary lattice is specified by a matrix A and a modulus q. Therefore, to use these lattices in practical constructions, we need algorithms to compute their basis matrices from A and q. Such bases can be obtained using standard integer linear algebra techniques, particularly the Hermite Normal Form (HNF). For example, a basis of the image lattice $\Lambda_q(A)$ can be computed by applying HNF to the matrix $[A^T \mid qI_m]$. Similarly, a basis of

the kernel lattice $\Lambda_q^\perp(A)$ can be computed using integer kernel algorithms based on HNF or SNF. These algorithms run in polynomial time in the input bit length. Note that a lattice basis is not unique, and different algorithms may produce different basis matrices for the same lattice.

6 The Short Integer Solution Problem

The Short Integer Solution (SIS) problem is a central computational problem underlying many lattice-based cryptographic constructions, including hash functions and digital signatures.

SIS problem

Let $q \geq 2$, let $A \in \mathbb{Z}_q^{n\times m}$ be a given matrix, and let $\beta > 0$ be a norm bound. The SIS problem asks to find a nonzero vector

$$\boldsymbol{x} \in \mathbb{Z}^m$$

such that

$$\boxed{A\boldsymbol{x} = \boldsymbol{0} \pmod q, \qquad 0 < \|\boldsymbol{x}\|_2 \leq \beta.}$$

In other words, SIS asks for a short, nonzero vector in the lattice $\Lambda_q^\perp(A)$.

6.1 Why is SIS a lattice problem?

By definition,

$$\Lambda_q^\perp(A) = \{\boldsymbol{x} \in \mathbb{Z}^m : A\boldsymbol{x} = \boldsymbol{0} \pmod q\}.$$

The SIS solution must satisfy the same modular equation, together with a shortness condition. Therefore, SIS is precisely the problem of finding a short, nonzero vector in a particular q-ary lattice.

6.2 A simple example

Consider

$$q = 5, \qquad A = \begin{pmatrix}1 & 2\end{pmatrix}.$$

The vector

$$\boldsymbol{x} = \begin{pmatrix}1\\2\end{pmatrix}$$

satisfies

$$A\boldsymbol{x} = 1 + 2(2) = 5 = 0 \pmod 5.$$

Its Euclidean norm is

$$\|\boldsymbol{x}\|_2 = \sqrt{1^2 + 2^2} = \sqrt{5}.$$

Thus, for any bound $\beta \geq \sqrt{5}$, this vector is a valid SIS solution.

Remark

The example illustrates correctness, not hardness. Cryptographic parameters (n, m, q, β) are chosen so that finding a sufficiently short, nonzero solution is computationally infeasible for an adversary, under an appropriate SIS hardness assumption.

6.3 Relation to Approximate SVP.

The SIS problem is closely related to the γ-Approximate Shortest Vector Problem (γ-SVP). Recall that γ-SVP asks to find a nonzero lattice vector $\boldsymbol{v}$ satisfying

$$\|\boldsymbol{v}\|_2 \leq \gamma\lambda_1(\mathcal{L}),$$

where $\lambda_1(\mathcal{L})$ denotes the length of the shortest nonzero vector in the lattice $\mathcal{L}$. Since SIS asks for a nonzero vector in $\Lambda_q^\perp(A)$ whose norm is at most β, it can be viewed as a bounded short-vector problem

on this particular q-ary lattice. However, SIS and γ-SVP are not identical: SIS specifies an absolute norm bound β, whereas γ-SVP specifies a bound relative to the shortest vector length. Under suitable parameter choices, worst-case lattice problems, including approximate SVP variants, can be reduced to average-case SIS instances. This connection provides the theoretical foundation for the hardness of SIS-based cryptographic constructions.

7 Ajtai's Lattice-Based Hash Function

Ajtai's hash function is an early lattice-based construction of a collision-resistant hash function. Its security is based on the hardness of the Short Integer Solution (SIS) problem. Unlike conventional hash functions, which are typically based on assumptions about the underlying hash algorithm, Ajtai's construction connects collision resistance to worst-case lattice problems through worst-case to average-case reductions.

7.1 Construction

Let $q \geq 2$ be an integer, and let

$$A \in \mathbb{Z}_q^{n \times m}$$

be a uniformly random matrix. The matrix A is the public key of the hash function. Consider the input domain $\{0,1\}^m$ and the output space $\mathbb{Z}_q^n$.

The hash function is defined as

$$\boxed{H_A : \{0,1\}^m \longrightarrow \mathbb{Z}_q^n, \qquad H_A(\boldsymbol{x}) = A\boldsymbol{x} \bmod q.}$$

In other words, the input is treated as a binary vector, and the hash value is obtained by multiplying it with the public matrix A, followed by reduction modulo q.

7.2 Algorithm

The construction consists of two algorithms: parameter generation and hash evaluation.

Algorithm 1: Ajtai's Lattice-Based Hash Function

Input: Security parameters n, m, q

Output: Public hash matrix A

1 **Parameter Generation:**;

2 Sample a uniformly random matrix

$$A \xleftarrow{\$} \mathbb{Z}_q^{n \times m}.$$

Publish A as the hash function parameter.

3 **Hash Evaluation:**;

Input: Public matrix A, input $\boldsymbol{x} \in \{0,1\}^m$

Output: Hash value $\boldsymbol{y} \in \mathbb{Z}_q^n$

4 Compute

$$\boldsymbol{y} = A\boldsymbol{x} \bmod q.$$

return $\boldsymbol{y}$;

7.3 Input and Output Length

The hash function maps an m-bit input message to a vector in $\mathbb{Z}_q^n$. Since each coordinate of the output can take one of q possible values, the total number of possible hash outputs is q^n. Therefore, the output can represent at most

$$\log_2(q^n) = n \log_2 q$$

bits.

In contrast, the input space $\{0,1\}^m$ contains 2^m possible messages. To obtain a compressing hash function, the number of possible inputs must be greater than the number of possible outputs:

$$2^m > q^n.$$

Taking logarithms on both sides gives

$$\boxed{m > n \log_2 q.}$$

Thus, the construction maps an m-bit message to an output of approximately $n \log_2 q$ bits, with $m > n \log_2 q$ ensuring compression. By the pigeonhole principle, this also guarantees that collisions exist. The security objective is to make finding such a collision computationally infeasible.

7.4 Why is the Hash Function Collision Resistant?

Suppose an adversary finds two distinct inputs $\boldsymbol{x}, \boldsymbol{x}' \in \{0,1\}^m$ such that

$$H_A(\boldsymbol{x}) = H_A(\boldsymbol{x}').$$

By the definition of the hash function,

$$A\boldsymbol{x} = A\boldsymbol{x}' \pmod q.$$

Rearranging, we obtain

$$A(\boldsymbol{x} - \boldsymbol{x}') = \mathbf{0} \pmod q.$$

Let

$$\boldsymbol{z} = \boldsymbol{x} - \boldsymbol{x}'.$$

Since $\boldsymbol{x} \neq \boldsymbol{x}'$, we have $\boldsymbol{z} \neq \mathbf{0}$. Furthermore, because both inputs are binary vectors,

$$z_i \in \{-1, 0, 1\}.$$

Consequently,

$$\|\boldsymbol{z}\|_2 \leq \sqrt{m}.$$

Therefore, a collision gives us a nonzero vector $\boldsymbol{z}$ satisfying

$$\boxed{A\boldsymbol{z} = \mathbf{0} \pmod q, \qquad 0 < \|\boldsymbol{z}\|_2 \leq \sqrt{m}.}$$

This is a valid solution to the SIS problem with norm bound $\beta = \sqrt{m}$.

Thus, if the corresponding SIS problem is computationally hard for a uniformly random matrix A, finding a collision in H_A is also computationally hard.

Note: The construction is collision resistant under the appropriate SIS hardness assumption and parameter choices. To obtain a compressing hash function, the parameters should also satisfy $2^m > q^n$, so that the input space is larger than the output space. This ensures that collisions exist, although finding one should remain computationally infeasible.

7.5 Illustrative Example

Consider the following small example:

$$q = 5, \qquad A = \begin{pmatrix} 1 & 2 \end{pmatrix}.$$

Let the two binary inputs be

$$\boldsymbol{x} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad \boldsymbol{x}' = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Their hash values are

$$H_A(\boldsymbol{x}) = 1, \qquad H_A(\boldsymbol{x}') = 2.$$

Thus, these inputs do not form a collision.

Now consider

$$\boldsymbol{x} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad \boldsymbol{x}' = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

We obtain

$$H_A(\boldsymbol{x}) = 1 + 2 = 3 \pmod 5, \qquad H_A(\boldsymbol{x}') = 0.$$

These are also not a collision.

This small example illustrates how the hash function is evaluated. Actual cryptographic parameters are much larger and are selected to ensure that finding a collision is computationally infeasible.

8 Regev's Lattice-Based One-Way Function

Regev's one-way function is based on the Learning With Errors (LWE) problem. Unlike Ajtai's hash function, which relies on the hardness of SIS, this construction uses a secret vector and a small error vector. The central idea is to compute a modular matrix product and then add a small random error. Recovering a valid input from the output is computationally difficult under the LWE assumption.

8.1 Learning With Errors (LWE)

The previously discussed problems, such as SVP, CVP, and SIVP, are primarily geometric problems defined over lattices. Although they provide important hardness foundations, directly expressing cryptographic operations using these geometric problems is not always convenient.

The Learning With Errors (LWE) problem provides an algebraic formulation based on noisy linear equations over the ring $\mathbb{Z}_q$. The central idea is to recover an unknown secret vector from several linear equations in which small random errors have been introduced.

Learning With Errors (LWE)

Let n and q be positive integers, and let χ be an error distribution over $\mathbb{Z}$. Choose a secret vector

$$\boldsymbol{s} \xleftarrow{\$} \mathbb{Z}_q^n.$$

An LWE sample is generated as follows:

$$\boldsymbol{a} \xleftarrow{\$} \mathbb{Z}_q^n, \qquad e \xleftarrow{\$} \chi,$$

and

$$\boxed{b = \langle \boldsymbol{a}, \boldsymbol{s} \rangle + e \pmod q.}$$

The resulting sample is the pair $(\boldsymbol{a}, b) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$.

Here, $\boldsymbol{a}$ is a uniformly random vector, $\boldsymbol{s}$ is the secret vector, and e is a small random error drawn from the error distribution χ.

Given multiple LWE samples

$$(\boldsymbol{a}_1, b_1), (\boldsymbol{a}_2, b_2), \ldots, (\boldsymbol{a}_m, b_m),$$

the objective is to recover the secret vector $\boldsymbol{s}$.

Matrix Representation. The collection of m LWE samples can be represented compactly in matrix form. Let

$$A = \begin{pmatrix} \boldsymbol{a}_1^T \\ \boldsymbol{a}_2^T \\ \vdots \\ \boldsymbol{a}_m^T \end{pmatrix} \in \mathbb{Z}_q^{m \times n}, \qquad \boldsymbol{e} = \begin{pmatrix} e_1 \\ e_2 \\ \vdots \\ e_m \end{pmatrix}, \qquad \boldsymbol{b} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}.$$

The LWE samples can then be expressed as

$$\boxed{\boldsymbol{b} = A\boldsymbol{s} + \boldsymbol{e} \pmod q.}$$

Search and Decision LWE There are two commonly studied variants of LWE:

1. **Search LWE:** Given sufficiently many LWE samples, recover the secret vector $\boldsymbol{s}$.
2. **Decision LWE:** Given several pairs $(\boldsymbol{a}_i, b_i)$, determine whether they are genuine LWE samples or uniformly random pairs from $\mathbb{Z}_q^n \times \mathbb{Z}_q$.

Why is LWE important? The introduction of noise makes recovering the secret difficult. Without the error term, the equations would form a system of linear equations modulo q, which could be solved efficiently using standard linear algebraic techniques.

However, with appropriately chosen error distributions and parameters, recovering the secret becomes computationally difficult.

8.2 Construction OWF

Let $q \geq 2$ be an integer, and let

$$A \in \mathbb{Z}_q^{n \times m}$$

be a uniformly random public matrix. The input consists of two vectors:

$$\boldsymbol{s} \xleftarrow{\$} \mathbb{Z}_q^n, \qquad \boldsymbol{e} \xleftarrow{\$} \chi^m,$$

where χ is a probability distribution over small integers, such as a discrete Gaussian distribution.

The one-way function is defined as

$$\boxed{g_A : \mathbb{Z}_q^n \times \mathrm{Supp}(\chi)^m \longrightarrow \mathbb{Z}_q^m, \qquad g_A(\boldsymbol{s}, \boldsymbol{e}) = A^T \boldsymbol{s} + \boldsymbol{e} \pmod q.}$$

Here, $\boldsymbol{s}$ is the secret vector and $\boldsymbol{e}$ is the error vector. The function is deterministic once the input pair $(\boldsymbol{s}, \boldsymbol{e})$ is fixed.

Algorithm The construction consists of two algorithms: parameter generation and function evaluation.

Algorithm 2: Regev's LWE-Based One-Way Function

Input: Parameters n, m, q, error distribution χ, input $(\boldsymbol{s}, \boldsymbol{e})$
Output: Public matrix A and output $\boldsymbol{y}$

1 Sample $A \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$ uniformly at random;
2 Compute $\boldsymbol{y} = A^T \boldsymbol{s} + \boldsymbol{e} \pmod q$;
3 **return** $(A, \boldsymbol{y})$;

Algorithm

8.3 Why is the Function One-Way?

Suppose an adversary is given the public matrix A and the output

$$\boldsymbol{y} = A^T \boldsymbol{s} + \boldsymbol{e} \pmod q.$$

The adversary's task is to recover a valid input pair $(\boldsymbol{s}, \boldsymbol{e})$ satisfying

$$\boxed{\boldsymbol{y} = A^T \boldsymbol{s} + \boldsymbol{e} \pmod q, \qquad \boldsymbol{e} \in \mathrm{Supp}(\chi)^m.}$$

This is precisely the Search-LWE problem in matrix form. To see this, write the columns of A as

$$A = [\boldsymbol{a}_1 \mid \boldsymbol{a}_2 \mid \cdots \mid \boldsymbol{a}_m].$$

Then the output can be expressed coordinate-wise as

$$y_i = \langle \boldsymbol{a}_i, \boldsymbol{s} \rangle + e_i \pmod q, \qquad 1 \leq i \leq m.$$

Thus, each coordinate of $\boldsymbol{y}$ represents an LWE sample.

If an adversary can efficiently invert the function by finding a valid preimage, it can solve the corresponding Search-LWE instance. Therefore, under the appropriate LWE hardness assumption and parameter choices, the function is one-way.

9 Regev's LWE-Based Public-Key Encryption

A public-key encryption (PKE) scheme requires two keys: a public key and a private key. The public key is shared with anyone who wants to encrypt a message, whereas the private key is kept secret and is used for decryption.

The main idea behind Regev's LWE-based encryption scheme is to introduce a small random error into a linear equation. During encryption, the message is embedded in the resulting noisy value. During decryption, the receiver uses the secret key to cancel the secret-dependent linear component, leaving only the small error and the encoded message.

The error is chosen sufficiently small so that, even after considering arithmetic modulo q, the resulting value remains close to the intended message encoding. In particular, the two possible encodings of a bit, 0 and $q/2$, remain distinguishable. This allows the decryptor to recover the original message.

9.1 Construction

Let q be an even integer, and let χ be an error distribution over small integers. The scheme consists of three algorithms: Key Generation, Encryption, and Decryption.

1. Key Generation Choose the public parameters

$$A \xleftarrow{\$} \mathbb{Z}_q^{n\times m}, \qquad \boldsymbol{s} \xleftarrow{\$} \mathbb{Z}_q^n, \qquad \boldsymbol{e} \xleftarrow{\$} \chi^m.$$

Compute

$$\boldsymbol{b} = A^T\boldsymbol{s} + \boldsymbol{e} \pmod q.$$

The keys are defined as follows:

$$\boxed{PK = (A, \boldsymbol{b}), \qquad SK = \boldsymbol{s}.}$$

Here, PK is the public key and SK is the private key.

2. Encryption To encrypt a message bit $\mu \in \{0, 1\}$, the sender performs the following steps:

1. Choose a random binary vector
$$\boldsymbol{x} \xleftarrow{\$} \{0,1\}^m.$$
2. Compute
$$\boldsymbol{u} = A\boldsymbol{x} \pmod q.$$
3. Compute
$$v = \boldsymbol{b}^T\boldsymbol{x} + \mu\frac{q}{2} \pmod q.$$

The resulting ciphertext is

$$\boxed{CT = (\boldsymbol{u}, v).}$$

3. Decryption Given the ciphertext $CT = (\boldsymbol{u}, v)$, the receiver uses the private key $\boldsymbol{s}$ to compute

$$\begin{aligned} d &= v - \boldsymbol{s}^T \boldsymbol{u} \pmod q \\ &= \boldsymbol{b}^T \boldsymbol{x} + \mu \frac{q}{2} - \boldsymbol{s}^T A \boldsymbol{x} \pmod q \\ &= (A^T \boldsymbol{s} + \boldsymbol{e})^T \boldsymbol{x} + \mu \frac{q}{2} - \boldsymbol{s}^T A \boldsymbol{x} \pmod q \\ &= \boldsymbol{e}^T \boldsymbol{x} + \mu \frac{q}{2} \pmod q. \end{aligned}$$

Since $\boldsymbol{e}$ is small and $\boldsymbol{x}$ is binary, the value $\boldsymbol{e}^T \boldsymbol{x}$ is also small. The receiver therefore recovers the message by checking which of the two values, 0 or $q/2$, is closer to d modulo q.

More precisely, let $\widetilde{d}$ be the centered representative of d in the interval $[-q/2, q/2)$. Then:

$$\boxed{\mu = \begin{cases} 0, & |\widetilde{d}| < q/4, \\ 1, & |\widetilde{d}| > q/4. \end{cases}}$$

The equality case can be excluded by choosing parameters that guarantee the error is strictly smaller than $q/4$.

9.2 Choosing the Error Size

The error vector $\boldsymbol{e}$ is sampled from a small-error distribution, typically a discrete Gaussian distribution. Its size must satisfy two requirements:

1. **Correctness:** The accumulated error during encryption must remain smaller than the decoding threshold $q/4$.
2. **Security:** The error must be sufficiently large to prevent an adversary from efficiently recovering the secret vector using the LWE samples.

Suppose each error coordinate is bounded by

$$|e_i| \le B,$$

where B is an integer. Since $\boldsymbol{x} \in \{0,1\}^m$, we have

$$|\boldsymbol{e}^T \boldsymbol{x}| \le \sum_{i=1}^{m} |e_i| \le mB.$$

Therefore, a sufficient condition for correct decryption is

$$\boxed{mB < \frac{q}{4}.}$$

If the error is represented using approximately k bits of magnitude, then B is of order 2^k. Consequently, the correctness condition becomes

$$\boxed{m2^k < \frac{q}{4},}$$

or equivalently,

$$k < \log_2 \left(\frac{q}{4m} \right).$$

In practice, the error is sampled from a discrete Gaussian distribution rather than a uniformly bounded distribution. Hence, parameters are selected so that the accumulated error exceeds the decoding threshold only with negligible probability. The error size must also satisfy the security requirements of the chosen LWE parameters.

9.3 Why Does Decryption Work?

Recall that the decrypted value is

$$d = \boldsymbol{e}^T \boldsymbol{x} + \mu \frac{q}{2} \pmod{q}.$$

There are two possible cases:

- If $\mu = 0$, then
$$d = \boldsymbol{e}^T \boldsymbol{x} \pmod{q}.$$
Thus, d remains close to 0.
- If $\mu = 1$, then
$$d = \boldsymbol{e}^T \boldsymbol{x} + \frac{q}{2} \pmod{q}.$$
Thus, d remains close to $q/2$ on the circle of residues modulo q.

For correct decryption, we require

$$\boxed{|\boldsymbol{e}^T \boldsymbol{x}| < \frac{q}{4}.}$$

This ensures that the error cannot move the encoded message from one decoding region into the other.

For example, if every error coordinate satisfies $|e_i| \le B$, then, since $x_i \in \{0, 1\}$,

$$|\boldsymbol{e}^T \boldsymbol{x}| \le mB.$$

Therefore, a sufficient condition for correct decryption is

$$mB < \frac{q}{4}.$$

For an unbounded error distribution such as a discrete Gaussian, parameters are selected so that this condition holds with overwhelming probability.

9.4 Illustrative Example

Consider a small example with

$$q = 20, \qquad \boldsymbol{e}^T \boldsymbol{x} = 2.$$

The two possible message encodings are 0 and $q/2 = 10$.

- For $\mu = 0$, the decrypted value is
$$d = 0 + 2 = 2 \pmod{20}.$$
As $2 < q/4 = 5$, the receiver decodes $\mu = 0$.
- For $\mu = 1$, the decrypted value is
$$d = 10 + 2 = 12 \pmod{20}.$$
Its centered representative is -8. Since $|-8| > 5$, the receiver decodes $\mu = 1$.

Thus, even though a small error is introduced during encryption, the receiver can recover the original message using the secret key.

9.5 Security of the Construction

The public key contains

$$(A, \boldsymbol{b}), \qquad \boldsymbol{b} = A^T \boldsymbol{s} + \boldsymbol{e} \pmod{q}.$$

Notice that the columns of A together with the corresponding entries of $\boldsymbol{b}$ form m LWE samples with the same secret vector $\boldsymbol{s}$.

An adversary who does not know $\boldsymbol{s}$ cannot efficiently remove the secret-dependent component from the public-key values. Under the appropriate Decision-LWE hardness assumption and parameter choices, the public key is computationally indistinguishable from a uniformly random pair of the same dimensions. This provides the basis for the semantic security of the encryption scheme against chosen-plaintext attacks (IND-CPA).